

REVERSE ENGINEERING // MASTERY COURSE

PHASE 1

Assembly Language

Machine ki bhasha — registers, instructions, stack aur function calls.

```
re@learning: ~/phase-1
$ ./start_learning.sh --phase 1
# Hinglish mein, simple examples ke saath – ek-ek step
$ status: READY ✓
```

 Detailed Theory

 Code + Line-by-line

 Interview Q&A

IN Hinglish

Reverse Engineering Mastery 2024 • A Self-Study Roadmap • Phase 1 Handbook

Phase 1 — Assembly Language (Machine Ki Bhasha)

Goal: Assembly padhna aur samajhna seekhna. Jab bhi tum `.exe` khologe, yahi dikhegi. Ye **sabse important phase** hai — isko achhe se karo. Time: **3-4 hafte**. Jaldbaazi bilkul nahi.

1.0 — Assembly Kyun Itni Important?

Jab ek program compile hota hai, wo machine code ban jaata hai. Ghidra/IDA jaise tools us machine code ko **Assembly** mein dikhte hain. Matlab jo bhi program tum reverse karoge, uski asli bhasha Assembly hi hogi.

Ek line yaad rakho: **"RE aati hai jab Assembly aati hai."** Baaki sab tools aur tricks bekaar hain agar tum assembly nahi padh sakte.

Ghabrao mat — assembly padhna seekhna hai, likhna nahi (abhi). Aur ye utni mushkil nahi jitni lagti hai, kyunki har instruction bahut chhota kaam karti hai.

1.1 — Assembly Hai Kya?

Assembly, machine code ka human-readable version hai. CPU ko diye jaane wale chhote-chhote commands (instructions).

Example:

```
mov eax, 5      ; eax naam ke dabbe mein 5 daalo
add eax, 3      ; eax mein 3 aur jodo (ab eax = 8)
```

- C ki ek line = kai assembly lines. Assembly bahut "low-level" hai — matlab CPU ke bahut kareeb.
- Har line ek chhota specific kaam karti hai: value move karo, jodo, compare karo, jump karo.

Syntax ke do styles (dhyan rakho)

- **Intel syntax:** `mov eax, 5` (destination pehle) — Windows/IDA/Ghidra mein common. **Hum yahi use karenge.**

- **AT&T syntax:** `mov $5, %eax` (source pehle, `%` aur `$` ke saath) — Linux/GDB default.

Confusion se bachne ke liye ek hi style pe focus karo. Intel syntax RE mein zyada common hai.

1.2 — CPU Registers (Chhote Dabbe)

Registers = CPU ke andar bahut chhote, bahut fast storage boxes jahan calculation hoti hai. CPU pehle data ko register mein laata hai, phir kaam karta hai.

General Purpose Registers (32-bit / 64-bit)

32-bit	64-bit	Aam kaam
<code>eax</code>	<code>rax</code>	Calculation, function ka return value yahan aata hai
<code>ebx</code>	<code>rbx</code>	General data
<code>ecx</code>	<code>rcx</code>	Loop counter (counting)
<code>edx</code>	<code>rdx</code>	Data, multiplication/division mein help
<code>esi</code>	<code>rsi</code>	Source (copy karte waqt "kahan se")
<code>edi</code>	<code>rdi</code>	Destination (copy karte waqt "kahan")

Special Registers

Register	Kaam
<code>esp</code> / <code>rsp</code>	Stack Pointer — stack ke top (upar) ko point karta hai
<code>ebp</code> / <code>rbp</code>	Base Pointer — current function ke stack frame ka base
<code>eip</code> / <code>rip</code>	Instruction Pointer — abhi kaunsi instruction chal rahi hai (★ bahut important)

EIP/RIP: Ye batata hai CPU abhi kahan hai. Agar hum ise control kar lein (exploit mein), toh program hijack ho jaata hai. Isliye ye register RE aur hacking dono mein star hai.

Register ke chhote hisse

Ek 64-bit register ke andar chhote versions bhi hote hain:

```
RAX (64-bit) — poora 8 bytes
└─ EAX (32-bit) — niche ke 4 bytes
    └─ AX (16-bit) — niche ke 2 bytes
        └─ AH | AL (8-bit) — AL sabse niche ka 1 byte
```

Jab code mein `al` dikhe, wo `eax` ka sabse chhota (1 byte) hissa hai. Ye choti values (jaise ek character) ke liye use hota hai.

1.3 — Common Instructions (Ratne nahi, samajhne hain)

Ye lists dekh ke ghabrana nahi — practice se sab yaad ho jaayega.

Data Move

```
mov eax, 5      ; eax = 5 (value copy)
mov ebx, eax    ; ebx = eax (ek register se doosre mein)
mov [0x1000], eax ; memory address 0x1000 pe eax ki value likho
mov eax, [0x1000] ; address 0x1000 se value eax mein laao
lea eax, [ebx+4] ; eax = ebx+4 ka ADDRESS (value nahi) — "load effective address"
```

`[ ]` brackets ka matlab: "is address pe jaake dekho/likho" (pointer jaisa). Bina bracket = seedha value ya register.

Arithmetic (ganit)

```
add eax, 3      ; eax = eax + 3
sub eax, 1      ; eax = eax - 1
inc eax         ; eax = eax + 1 (increment)
dec eax         ; eax = eax - 1 (decrement)
imul eax, 2     ; eax = eax * 2 (multiply)
xor eax, eax    ; eax = 0 (khud ke saath XOR = 0, ye common trick hai!)
```

`xor eax, eax` register ko 0 karne ka fast tarika hai. RE mein baar-baar dikhega — ise "eax = 0" samajhna.

Compare & Jump (🌟 decisions yahan hote hain)

```
cmp eax, 10      ; eax aur 10 compare karo (result flags mein)
je label         ; Jump if Equal (agar barabar the)
jne label        ; Jump if Not Equal
jg label         ; Jump if Greater
jl label         ; Jump if Less
jmp label        ; hamesha jump (bina condition)
```

Ye if-else banate hain! `cmp` do cheezein compare karta hai, phir `je` / `jne` /etc. decide karte hain kahan jaana hai.

Function Calls

```
call func        ; 'func' function ko bulao (aur wapas aane ka address yaad rakho)
ret              ; function se wapas jao (return)
push eax         ; eax ki value stack pe daalo (upar)
pop  eax         ; stack se upar wali value eax mein nikaalo
```

Cheat Sheet Table

Instruction	Kaam	Yaad rakhne ka tarika
<code>mov</code>	copy data	"move"
<code>add</code> / <code>sub</code>	jodna/ghatana	+ / -
<code>cmp</code>	compare	do cheez check
<code>jmp</code>	jump (always)	seedha kudo
<code>je</code> / <code>jne</code>	jump if equal/not	condition pe kudo
<code>call</code>	function bulao	function call
<code>ret</code>	wapas	return
<code>push</code> / <code>pop</code>	stack pe daalo/nikaalo	stack
<code>xor r,r</code>	register = 0	zero karne ka trick
<code>lea</code>	address nikaalo	pointer math

1.4 — Ek Real Example: C ↔ Assembly

Chalo ek chhota C program lete hain aur uski assembly samajhte hain.

C Code

```
int main() {
    int a = 5;
    int b = 3;
    int c = a + b;    // c = 8
    return c;
}
```

Assembly (approx, Intel syntax)

```
main:
    push ebp          ; purana base pointer bachao (stack pe)
    mov  ebp, esp      ; naya stack frame set karo
    mov  dword [ebp-4], 5 ; a = 5 (stack pe rakha)
    mov  dword [ebp-8], 3 ; b = 3
    mov  eax, [ebp-4]   ; eax = a (5)
    add  eax, [ebp-8]   ; eax = eax + b = 5+3 = 8
    mov  [ebp-12], eax  ; c = eax = 8
    mov  eax, [ebp-12]  ; return value eax mein daalo (8)
    pop  ebp           ; base pointer wapas
    ret               ; return
```

Line by line samjho: - `push ebp` / `mov ebp, esp` — har function shuru mein ye karta hai ("prologue"). Purana frame bachata hai, naya banata hai. - `mov dword [ebp-4], 5` — variable `a` stack pe `ebp-4` address pe hai, usme 5 daala. (`dword` = 4 bytes ka int) - `mov eax, [ebp-4]` — `a` ki value `eax` mein laayi. - `add eax, [ebp-8]` — usme `b` jod diya. Ab `eax = 8`. - `mov eax, [ebp-12]` — return karne ke liye value `eax` mein rakhi (return value hamesha `eax/rax` mein). - `pop ebp` / `ret` — frame wapas, function khatam ("epilogue").

Dekha? C ki 4 lines = assembly ki ~10 lines. Har local variable stack pe `[ebp-kuch]` pe rehta hai. Ye pattern har jagah dikhega — ise pehchano.

If-else assembly mein kaisa dikhta hai

```
if (x == 10) { y = 1; }  
else      { y = 2; }
```

```
    cmp dword [ebp-4], 10    ; x ko 10 se compare  
    jne else_part          ; agar equal NAHI, else pe jao  
    mov dword [ebp-8], 1    ; y = 1 (if wala part)  
    jmp end  
else_part:  
    mov dword [ebp-8], 2    ; y = 2 (else wala part)  
end:
```

Pattern: `cmp` + conditional jump = if-else. Ye pehchan gaye toh RE bahut aasan.

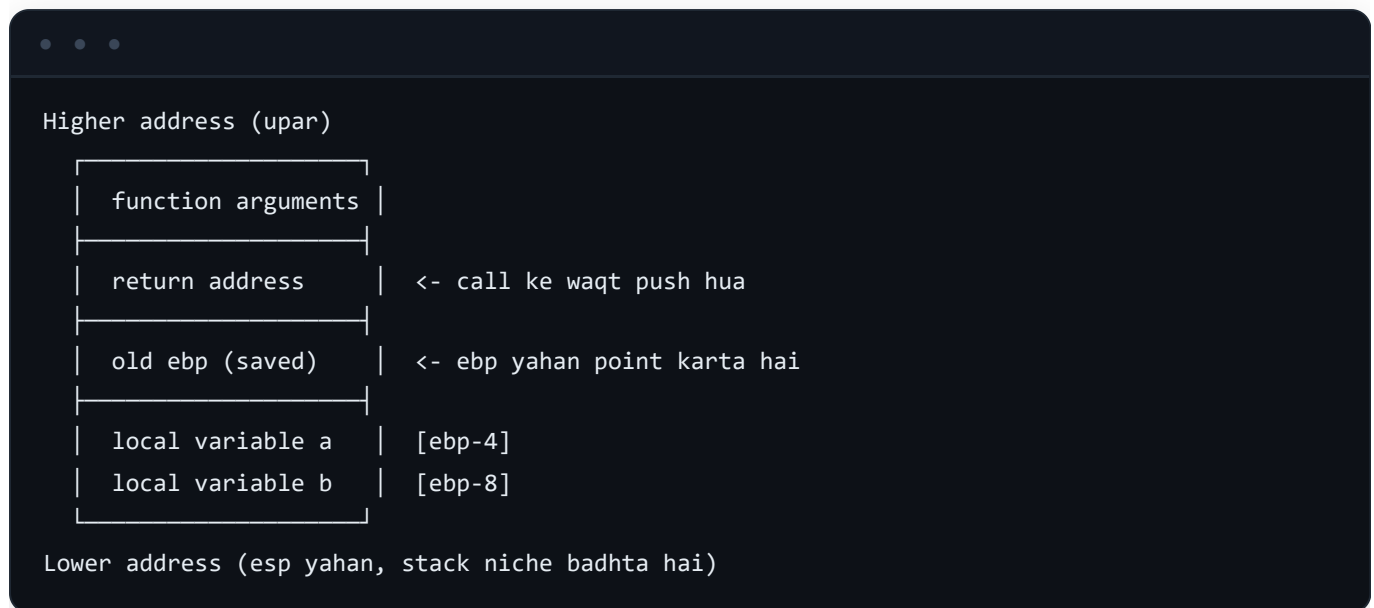
1.5 — Stack Ko Deeply Samjho (★ ★)

Stack samajhna RE ka bada milestone hai. Stack ek "plate ka dher" jaisa hai — LIFO (Last In, First Out). Jo aakhri mein rakha, wahi pehle nikalta hai.

Stack kya rakhta hai?

- Function ke **local variables**
- Function ke **arguments** (parameters)
- **Return address** (function khatam hone pe kahan wapas jaana hai)
- Purana **base pointer** (ebp)

Stack Frame ka structure (ek function ka)



Important: Stack **niche ki taraf** badhta hai (higher se lower address). **push** karne pe esp chhota hota hai, **pop** pe bada. Ye ulta lagta hai but yaad rakho.

call aur ret internally kya karte hain

- call func** = pehle wapas aane ka address stack pe **push** karta hai, phir func pe jump.
- ret** = stack se wo address **pop** karta hai aur wahan wapas jump.

Isiliye **buffer overflow** attacks kaam karte hain — agar hum stack pe return address ko overwrite kar dein, toh program kahin aur "wapas" jaayega jahan hum chahein. (Ye advanced topic hai, abhi bas idea rakho.)

1.6 — x86 vs x64 (32-bit vs 64-bit)

	x86 (32-bit)	x64 (64-bit)
Registers	eax, ebx...	rax, rbx... (aur r8 - r15 extra)
Address size	4 bytes	8 bytes
Function args	zyada tar stack pe	pehle registers mein (rcx, rdx, r8, r9 Windows)
Seekhna	aasan (pehle karo)	modern (baad mein)

Advice: Pehle x86 (32-bit) achhe se samajho — simple hai. Phir x64 pe jaao. Concepts same hain, bas thoda badla hua.

1.7 — Calling Conventions (Functions ko args kaise milte hain)

Jab ek function ko call karte hain, arguments kaise pass hote hain — iske rules ko "calling convention" kehte hain.

- **x86 cdecl:** arguments stack pe push hote hain (ulte order mein). Return value `eax` mein.
- **x64 Windows:** pehle 4 args registers mein — `rcx, rdx, r8, r9`. Baaki stack pe. Return `rax` mein.
- **x64 Linux (System V):** pehle 6 args — `rdi, rsi, rdx, rcx, r8, r9`.

RE mein jab `call` dikhe, uske pehle wale `mov rcx, ...` / `push ...` dekho — wahi function ke arguments hain. Return value hamesha `eax` / `rax` mein aati hai.

Phase 1 — Hands-On Practice

1. **godbolt.org** kholo. Chhote C programs likho, unki assembly dekho. C aur assembly ko side-by-side compare karo.
2. Ye programs try karo: simple addition, if-else, ek loop, ek function call.
3. Har assembly line ko English/Hinglish mein "translate" karo — ye best exercise hai.
4. Register kis line pe kya value rakhega — dimaag mein simulate karo.

Key Takeaways (Ek Line Mein)

1. Assembly = machine code ka human-readable version. **RE ki asli bhasha.**
2. **Registers** = CPU ke chhote fast dabbe. `eax` = calculation/return, `esp` = stack top, `eip` = abhi kahan.
3. `[ ]` brackets = memory access (pointer jaisa).
4. `cmp` + conditional jump (`je` / `jne`) = **if-else**. Loops mein bhi yahi.
5. **Stack niche badhta hai**, LIFO hai, aur function ke local vars + return address rakhta hai.
6. `call` return address push karta hai, `ret` use pop karke wapas jaata hai.
7. Return value hamesha `eax` / `rax` mein.

8. Pehle **x86** (aasan), phir **x64**.

✓ **Phase 1 Complete Jab:** Tum ek simple assembly code padh ke samajh sako ki wo kya kar raha hai — jaise loop, if-else, addition, function call. Ye aa gaya toh RE ka sabse bada hurdle paar. Ab tools seekhne ki baari (Phase 2)! 🚀

Phase 1 — Interview Questions & Answers (Hinglish)

Assembly Language se related important interview questions, simple Hinglish jawaab ke saath.

Q1. Assembly language kya hai?

Ans: Assembly, machine code ka human-readable version hai. CPU ko diye jaane wale chhote-chhote commands (instructions) jaise `mov eax, 5`. Har assembly instruction seedha ek machine code se map hoti hai. RE mein jab hum `.exe` kholte hain, tools use assembly mein dikhate hain.

Q2. Register kya hote hain?

Ans: Registers CPU ke andar bahut chhote aur bahut fast storage boxes hain jahan actual calculation hoti hai. CPU pehle data ko RAM se register mein laata hai, phir uspe kaam karta hai. Examples: `eax`, `ebx`, `ecx`, `esp`, `eip`.

Q3. EAX, ESP, aur EIP registers ka kya kaam hai?

Ans: - **EAX** — general calculations aur function ka return value yahan aata hai. - **ESP** (Stack Pointer) — stack ke top (upar) ko point karta hai. - **EIP** (Instruction Pointer) — batata hai CPU abhi kaunsi instruction chala raha hai. Ye bahut important hai — exploit mein isko control karna hi program hijack karna hota hai.

Q4. Intel aur AT&T syntax mein kya farak hai?

Ans: Dono assembly likhne ke styles hain. - **Intel:** `mov eax, 5` — destination pehle, source baad mein. Windows/IDA/Ghidra mein common. - **AT&T:** `mov $5, %eax` — source pehle, `%` aur `$` symbols ke saath. Linux/GDB mein default.

RE mein Intel syntax zyada use hota hai.

Q5. `mov eax, [0x1000]` aur `mov eax, 0x1000` mein kya farak hai?

Ans: Brackets `[ ]` ka matlab "memory access" hota hai. - `mov eax, 0x1000` — eax mein seedha value 0x1000 daal do. - `mov eax, [0x1000]` — address 0x1000 pe jo value rakhi hai, wo eax mein laao (pointer ki tarah).

Q6. `cmp` aur conditional jumps if-else kaise banate hain?

Ans: `cmp` do values ko compare karta hai aur result "flags" mein rakhta hai. Uske baad conditional jump (`je` = jump if equal, `jne` = jump if not equal, `jg` = greater, `jl` = less) decide karta hai kahan jaana hai. Ye combination hi assembly mein if-else aur loops banata hai.

Q7. Stack kya hai? Wo kis direction mein badhta hai?

Ans: Stack memory ka ek hissa hai jo LIFO (Last In First Out) tarike se kaam karta hai — jaise platon ka dher. Ye function ke local variables, arguments, return address, aur saved base pointer rakhta hai. Important baat: stack **niche ki taraf** (higher se lower address) badhta hai. `push` karne pe ESP chhota hota hai, `pop` pe bada.

Q8. `call` aur `ret` instructions internally kya karte hain?

Ans: `call func` pehle wapas aane ka address (return address) stack pe **push** karta hai, phir `func` pe jump karta hai. `ret` stack se wahi return address **pop** karke wapas us jagah jump kar jaata hai. Isliye function khatam hone pe program sahi jagah lautta hai.

Q9. Stack frame kya hota hai?

Ans: Har function call ka apna ek memory area stack pe hota hai — usi ko stack frame kehte hain. Ismein us function ke arguments, return address, saved old EBP, aur local variables hote hain. EBP (base pointer) frame ke base ko point karta hai, aur local variables usually `[ebp-4]`, `[ebp-8]` jaise access hote hain.

Q10. x86 aur x64 mein kya farak hai?

Ans: - **x86 (32-bit):** registers `eax, ebx...`, addresses 4 bytes, function arguments zyada tar stack pe pass hote hain. - **x64 (64-bit):** registers `rax, rbx...` plus extra `r8 - r15`, addresses 8 bytes, pehle kuch arguments registers mein pass hote hain (Windows: `rcx, rdx, r8, r9`).

Pehle x86 seekhna aasan hota hai, phir x64.

Q11. Calling convention kya hoti hai?

Ans: Calling convention wo rules hain jo batate hain ki function ko arguments kaise diye jaayenge aur return value kahan milegi. Example: x86 cdecl mein arguments stack pe push hote hain aur return value `eax` mein aati hai; x64 Windows mein pehle 4 args `rcx, rdx, r8, r9` registers mein jaate hain.

Q12. `xor eax, eax` kya karta hai aur kyun common hai?

Ans: Kisi register ko khud ke saath XOR karne pe wo **0** ho jaata hai. `xor eax, eax` matlab `eax = 0`. Compiler isko `mov eax, 0` ki jagah use karta hai kyunki ye thoda fast aur chhota hota hai. RE mein ye pattern bahut dikhega — ise "eax ko zero karna" samajhna.

Q13. `lea` instruction kya karti hai?

Ans: `lea` (Load Effective Address) kisi address ko calculate karke register mein daal deti hai — value nahi, sirf address. Jaise `lea eax, [ebx+4]` `eax` mein `ebx+4` ka address rakhega. Compiler ise pointer math aur kabhi-kabhi fast arithmetic ke liye bhi use karta hai.

Q14. Function ka return value kahan milta hai?

Ans: Convention ke hisaab se function ka return value hamesha accumulator register mein aata hai — x86 mein `eax`, x64 mein `rax`. RE mein jab `call` ke baad `eax` ki value use ho rahi ho, wo function ka result hai.

Q15. Register ke chhote hisse (jaise AL, AX) kya hote hain?

Ans: Ek bade register ke andar chhote versions hote hain. Jaise `RAX` (64-bit) ke andar `EAX` (32-bit), uske andar `AX` (16-bit), aur uske andar `AH / AL` (8-bit) hote hain. `AL` sabse niche ka 1 byte hai — chhoti values (jaise ek character) ke liye use hota hai.

Phase 1 — Interview Questions & Answers (English)

The same Assembly Language interview questions, answered in clear English for formal interviews.

Q1. What is assembly language?

Ans: Assembly is the human-readable form of machine code — small instructions given to the CPU, such as `mov eax, 5`. Each assembly instruction maps directly to machine code. When we open an `.exe` in RE tools, they present it as assembly.

Q2. What are registers?

Ans: Registers are very small, very fast storage locations inside the CPU where actual computation happens. The CPU loads data from RAM into a register, operates on it, and stores the result. Examples: `eax`, `ebx`, `ecx`, `esp`, `eip`.

Q3. What is the role of the EAX, ESP, and EIP registers?

Ans: - **EAX** — used for general calculations and holds a function's return value. - **ESP** (Stack Pointer) — points to the top of the stack. - **EIP** (Instruction Pointer) — indicates the instruction the CPU is currently executing. It's crucial: controlling EIP is what lets an attacker hijack program flow.

Q4. What is the difference between Intel and AT&T syntax?

Ans: They are two styles of writing assembly. - **Intel:** `mov eax, 5` — destination first, then source. Common on Windows/IDA/Ghidra. - **AT&T:** `mov $5, %eax` — source first, with `%` and `$` symbols. Default on Linux/GDB.

Intel syntax is more common in RE.

Q5. What is the difference between `mov eax, [0x1000]` and `mov eax, 0x1000` ?

Ans: Brackets `[ ]` mean memory access. - `mov eax, 0x1000` — put the literal value 0x1000 into eax. - `mov eax, [0x1000]` — load the value stored at memory address 0x1000 into eax (like dereferencing a pointer).

Q6. How do `cmp` and conditional jumps form if-else logic?

Ans: `cmp` compares two values and sets CPU flags based on the result. A following conditional jump (`je` = jump if equal, `jne` = jump if not equal, `jg` = greater, `jle` = less) then decides where to go. This combination implements if-else statements and loops in assembly.

Q7. What is the stack and in which direction does it grow?

Ans: The stack is a region of memory that works in LIFO (Last In, First Out) order, like a stack of plates. It stores a function's local variables, arguments, return address, and saved base pointer. Importantly, the stack grows **downward** (from higher to lower addresses): `push` decreases ESP, `pop` increases it.

Q8. What do the `call` and `ret` instructions do internally?

Ans: `call func` first **pushes** the return address onto the stack, then jumps to `func`. `ret` **pops** that return address off the stack and jumps back to it, so execution resumes right after the original call.

Q9. What is a stack frame?

Ans: Each function call has its own region on the stack called a stack frame. It contains that function's arguments, the return address, the saved old EBP, and its local variables. EBP (base pointer) points to the frame's base, and locals are typically accessed as `[ebp-4]`, `[ebp-8]`, etc.

Q10. What is the difference between x86 and x64?

Ans: - **x86 (32-bit):** registers `eax, ebx...`, 4-byte addresses, and function arguments mostly passed on the stack. - **x64 (64-bit):** registers `rax, rbx...` plus extra `r8 - r15`, 8-byte addresses, and the first few arguments passed in registers (Windows: `rcx, rdx, r8, r9`).

x86 is easier to learn first, then move to x64.

Q11. What is a calling convention?

Ans: A calling convention is the set of rules defining how arguments are passed to a function and where the return value is placed. For example, x86 cdecl pushes arguments on the stack and returns the value in `eax`; x64 Windows passes the first four arguments in `rcx, rdx, r8, r9`.

Q12. What does `xor eax, eax` do and why is it common?

Ans: XOR-ing a register with itself sets it to **0**, so `xor eax, eax` means `eax = 0`. Compilers use it instead of `mov eax, 0` because it's slightly faster and smaller in machine code. It's a very common pattern in RE — recognize it as "zeroing the register."

Q13. What does the `lea` instruction do?

Ans: `lea` (Load Effective Address) computes an address and stores it in a register — the address, not the value at it. For example, `lea eax, [ebx+4]` puts the address `ebx+4` into `eax`. Compilers use it for pointer arithmetic and sometimes for fast arithmetic.

Q14. Where is a function's return value stored?

Ans: By convention, the return value is placed in the accumulator register — `eax` on x86, `rax` on x64. In RE, if `eax` is used right after a `call`, that value is the function's result.

Q15. What are the smaller sub-registers like AL and AX?

Ans: A large register contains smaller sub-registers. `RAX` (64-bit) contains `EAX` (32-bit), which contains `AX` (16-bit), which contains `AH` / `AL` (8-bit). `AL` is the lowest byte — used for small values like a single character.